

9

SECURITY AUDITING



Intertwined with the access check process is the auditing process. An administrator can configure the system's auditing mechanism to generate a log of accessed resources. Each log event will include details about the user and application that opened the resource and whether the access succeeded or failed. This information can help us identify incorrect security settings or detect malicious access to sensitive resources.

In this short chapter, we'll first discuss where the resource access log gets stored once the kernel generates it. We'll then describe how a system administrator can configure the audit mechanism. Finally, we'll detail how to configure individual resources to generate audit log events through the SACL.

The Security Event Log

Windows generates log events whenever an access check succeeds or fails. The kernel writes these log events to the *security event log*, which only administrators can access.

When performing access checks on kernel resources, Windows will generate the following types of audit events. The security event log represents these by using the event ID included in parentheses:

- Object handle opened (4656)
- Object handle closed (4658)
- Object deleted (4660)
- Object handle duplicated (4690)
- SACL changed (4717)

When we access resources via kernel system calls such as `NtCreateMutant`, the auditing mechanism generates these events automatically. But for the object-related audit events, we must first configure two aspects of the system: we must set the system policy to generate audit events, and we must enable audit ACEs in the resource's SACL. Let's discuss each of these configuration requirements in turn.

Configuring the System Audit Policy

Most Windows users don't need to capture audit information for kernel resources, so the audit policy is disabled by default. Enterprise environments commonly configure the audit policy through a *domain security policy*, which the enterprise network distributes to the individual devices.

Users not in an enterprise network can enable the audit policy manually. One way to do so is to edit the *local security policy*, which looks the same as the domain security policy but applies only to the current system. There are two types of audit policy: the legacy policy used prior to Windows 7 and the advanced audit policy. Using the advanced audit policy is recommended, as it provides more granular configuration; we won't discuss the legacy policy further.

If you open the local security policy editor by running the `secpol.msc` command in PowerShell, you can view the current configuration of the advanced audit policy, as shown in **Figure 9-1**.

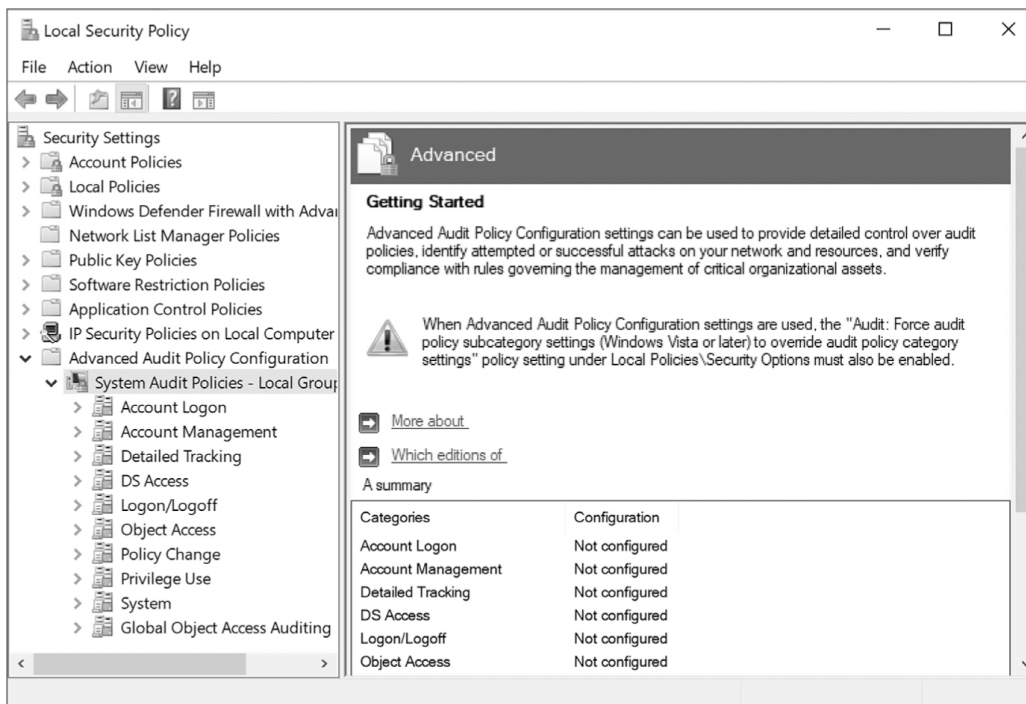


Figure 9-1: The security policy editor showing the advanced audit policy

As you can see, the categories in the audit policy aren't currently configured. To explore how audit events are generated, we'll use PowerShell to enable the required audit policy temporarily and run some example code. Any changes you make with PowerShell won't be reflected in the local security policy, which will revert the next time it synchronizes (for example, during a reboot or when the group policy is updated on an enterprise network). You can force the settings to synchronize by running the command `gpupdate.exe /force` as an administrator in PowerShell or at the command prompt.

Advanced audit policies have two levels: a top-level category and multiple subcategories. You can query for the top-level categories using `Get-NtAuditPolicy`, as in [Listing 9-1](#).

```
PS> Get-NtAuditPolicy
Name                               SubCategory Count
----
System                             5
Logon/Logoff                       11
Object Access                      14
Privilege Use                       3
Detailed Tracking                   6
Policy Change                       6
```

Account Management	6
DS Access	4
Account Logon	4

Listing 9-1: The top-level audit policy categories

In the output, you can see the name of each category and a count of its subcategories. Each category also has an associated GUID, but this value is hidden by default. To see it, select the `Id` property from the command's output, as shown in [Listing 9-2](#).

```
PS> Get-NtAuditPolicy | Select-Object Name, Id
Name                               Id
----                               --
System                             69979848-797a-11d9-bed3-505054503030
Logon/Logoff                       69979849-797a-11d9-bed3-505054503030
Object Access                      6997984a-797a-11d9-bed3-505054503030
--snip--
```

Listing 9-2: Displaying category GUIDs

You can display the subcategories by using the `ExpandCategory` parameter. In [Listing 9-3](#), we specify the `System` category by name and then expand the output to show its subcategories.

```
PS> Get-NtAuditPolicy -Category System -ExpandCategory
Name                               Policy
----                               -
Security State Change             Unchanged
Security System Extension         Unchanged
System Integrity                  Unchanged
IPsec Driver                     Unchanged
Other System Events              Unchanged
```

Listing 9-3: Displaying the audit policy's subcategories

You can also select a category by specifying its GUID using the `CategoryGuid` parameter. The audit policy is based on these subcategories. Each subcategory policy can have one or more of the following values:

Unchanged The policy is not configured and should not be changed.

Success The policy should generate audit events when an auditable resource is opened successfully.

Failure The policy should generate audit events when an auditable resource can't be opened.

None The policy should never generate an audit event.

In [Listing 9-3](#) the subcategories all show the value `Unchanged`, which means no policy has been configured. We can enable kernel object auditing by running the commands shown in [Listing 9-4](#) as an administrator.

```
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> Set-NtAuditPolicy -Category ObjectAccess -Policy Success,
Failure -PassThru
```

Name	Policy
----	-----
File System	Success, Failure
Registry	Success, Failure
Kernel Object	Success, Failure
SAM	Success, Failure
Certification Services	Success, Failure
Application Generated	Success, Failure
Handle Manipulation	Success, Failure
File Share	Success, Failure
Filtering Platform Packet Drop	Success, Failure
Filtering Platform Connection	Success, Failure
Other Object Access Events	Success, Failure
Detailed File Share	Success, Failure
Removable Storage	Success, Failure
Central Policy Staging	Success, Failure

Listing 9-4: Setting the policy and viewing the resulting ObjectAccess audit policy list

Here, we've enabled the `Success` and `Failure` audit policies for all subcategories under `ObjectAccess`. To make this modification, we need the `SeSecurityPrivilege` privilege. We can set a single subcategory rather than

the entire category by name by using the `SubCategoryName` parameter or specifying the GUID using `SubCategoryGuid`.

We confirm that the audit policy has been configured correctly by specifying the `PassThru` parameter, which lists the modified `SubCategory` objects. The output displays some important audit policies, including `File System`, `Registry`, and `Kernel Object`, which enable auditing on files, registry keys, and other kernel objects, respectively.

You can run the following command as an administrator to disable the change we made in [Listing 9-4](#):

```
PS> Set-NtAuditPolicy -Category ObjectAccess -Policy None
```

Unless you need to enable the audit policy for some reason, it's best to disable it once you're finished experimenting.

Configuring the Per-User Audit Policy

In addition to configuring a system-wide policy, it's also possible to configure the audit policy on a per-user basis. You could use this feature to add auditing to a specific user account in cases when the system does not define an overall audit policy. You could also use it to exclude a specific user account from auditing. To facilitate this behavior, the policy settings differ slightly for per-user policies:

Unchanged The policy is not configured. When set, the policy should not be changed.

SuccessInclude The policy should generate audit events on success, regardless of the system policy.

SuccessExclude The policy should never generate audit events on success, regardless of the system policy.

FailureInclude The policy should generate audit events on failure, regardless of the system policy.

FailureExclude The policy should never generate audit events on failure, regardless of the system policy.

None The policy should never generate an audit event.

To configure a per-user policy, you can specify a SID to the `User` parameter when using the `Set-NtAuditPolicy` command. This SID must represent a user account; it can't represent a group, such as *Administrators*, or a service account, such as *SYSTEM*, or you'll receive an error when setting the policy.

Listing 9-5 configures a per-user policy for the current user. You must run these commands as an administrator.

```
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> $sid = Get-NtSid
PS> Set-NtAuditPolicy -Category ObjectAccess -User $sid -UserPolicy
SuccessExclude
PS> Get-NtAuditPolicy -User $sid -Category ObjectAccess -ExpandCategory
```

Name	User	Policy
File System	GRAPHITE\admin	SuccessExclude
Registry	GRAPHITE\admin	SuccessExclude
Kernel Object	GRAPHITE\admin	SuccessExclude
SAM	GRAPHITE\admin	SuccessExclude

--snip--

Listing 9-5: Configuring a per-user audit policy

Here, we specify the user's SID to the `User` parameter, then specify the `SuccessExclude` user policy. This will exclude success audit events for only this user. If you want to remove the per-user policy for a user, you can specify the `None` user policy:

```
PS> Set-NtAuditPolicy -Category ObjectAccess -User $sid -UserPolicy None
```

You can also enumerate all users who have configured policies using the `AllUser` parameter of `Get-NtAuditPolicy`, as shown in **Listing 9-6**.

```
PS> Get-NtAuditPolicy -AllUser
Name           User           SubCategory Count
-----
System         GRAPHITE\admin 5
Logon/Logoff   GRAPHITE\admin 11
Object Access  GRAPHITE\admin 14
--snip--
```

Listing 9-6: Querying per-user policies for all users

You now know how to query and set policies for the system and for a specific user. Next, we'll look at how to grant users the access needed to query and set these policies on the system.

Audit Policy Security

To query or set a policy, the caller must have `SeSecurityPrivilege` enabled on their token. If the privilege is not enabled, LSASS will perform an access check based on a security descriptor in the system configuration. We can configure the following access rights in the security descriptor to grant a user the ability to query or set the policy for the system or a single user:

SetSystemPolicy Enables setting the system audit policy

QuerySystemPolicy Enables querying the system audit policy

SetUserPolicy Enables setting a per-user audit policy

QueryUserPolicy Enables querying a per-user audit policy

EnumerateUsers Enables enumerating all per-user audit policies

SetMiscPolicy Enables setting a miscellaneous audit policy

QueryMiscPolicy Enables querying a miscellaneous audit policy

No standard auditing API seems to use the `SetMiscPolicy` and `QueryMiscPolicy` access rights, but because they are defined in the Windows SDK, I've in-

cluded them here for completeness.

As an administrator, you can query the currently configured security descriptor by enabling `SeSecurityPrivilege` and using the `Get-NtAuditSecurity` command, as shown in [Listing 9-7](#).

```
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> $sd = Get-NtAuditSecurity
PS> Format-NtSecurityDescriptor $sd -Summary -MapGeneric
<DACL>
❶ BUILTIN\Administrators: (Allowed)(None)(GenericRead)
NT AUTHORITY\SYSTEM: (Allowed)(None)(GenericRead)
```

Listing 9-7: Querying and displaying the audit security descriptor

We pass the queried security descriptor to `Format-NtSecurityDescriptor` to display the DACL. Notice that only *Administrators* and *SYSTEM* can access the policy ❶. Also, they're limited to `GenericRead` access, which allows users to query the policy but not modify it. Thus, even administrators will need to enable `SeSecurityPrivilege` to modify the audit policy, as that privilege bypasses any access check.

NOTE

A user who has not been granted read access to the policy can still query the advanced audit categories and subcategories, which ignore the security descriptor. However, they won't be granted access to query the configured settings. `Get-NtAuditPolicy` will return the value of `Unchanged` for audit settings the user wasn't able to query.

If you want to allow non-administrators to change the advanced audit policy, you can change the security descriptor using the `Set-NtAuditSecurity` command. Run the commands in [Listing 9-8](#) as an administrator.

```
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> $sd = Get-NtAuditSecurity
PS> Add-NtSecurityDescriptorAce $sd -Sid "LA" -Access GenericAll
PS> Set-NtAuditSecurity $sd
```

Listing 9-8: Modifying the audit security descriptor

We first query the existing security descriptor for the audit policy and grant the local administrator all access rights. Then we set the modified security descriptor using the `Set-NtAuditSecurity` command. Now the local administrator can query and modify the audit policy without needing to enable `SeSecurityPrivilege`.

You shouldn't normally reconfigure the security of the audit policy, and you certainly shouldn't grant all users write access. Note that the security descriptor doesn't affect who can query or set the security descriptor itself; only callers with `SeSecurityPrivilege` enabled can do this, no matter the values in the security descriptor.

Configuring the Resource SACL

Just enabling the audit policies isn't enough to start generating audit events. We also need to configure an object's SACL to specify the auditing rules to use. To set the SACL on an object we'll again need to enable `SeSecurityPrivilege`, which can only be done as an administrator. [Listing 9-9](#) demonstrates the process for creating a `Mutant` object with a SACL.

```
PS> $sd = New-NtSecurityDescriptor -Type Mutant
PS> Add-NtSecurityDescriptorAce $sd -Type Audit -Access GenericAll
    -Flags SuccessfulAccess, FailedAccess -KnownSid World -MapGeneric
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> Clear-EventLog -LogName "Security"
PS> Use-NtObject($m = New-NtMutant "ABC" -Win32Path -SecurityDescriptor $sd) {
    Use-NtObject($m2 = Get-NtMutant "ABC" -Win32Path) {
    }
}
```

Listing 9-9: Creating a Mutant object with a SACL

We start by creating an empty security descriptor, then add a single `Audit` ACE to the SACL. Other ACE types we could add include `AuditObject` and `AuditCallback`.

The processing of Audit ACEs looks a lot like the discretionary access check described in [Chapter 7](#). The SID must match a group in the calling token (including any `DenyOnly` SIDs), and the access mask must match one or more bits of the granted access. The *Everyone* group's SID is a special case; it will always match, regardless of whether the SID is available in the token.

In addition to any of the usual inheritance ACE flags, such as `InheritOnly`, the Audit ACE must specify one or both of the `SuccessfulAccess` and `FailedAccess` flags, which provide the auditing code with the conditions in which it should generate the audit entry.

We'll create the `Mutant` object with a security descriptor containing the SACL. Before creating the object, we need to enable `SeSecurityPrivilege`. If we don't do this, the creation will fail. To make it easier to see the generated audit event, we also clear the security event log. Next, we create the object, passing it the SACL we built, and then reopen it to trigger the generation of an audit log.

Now we can query the security event log using `Get-WinEvent`, passing it the event ID 4656 to find the generated audit event ([Listing 9-10](#)).

```
PS> $filter = @{logname = 'Security'; id = @(4656)}
PS> Get-WinEvent -FilterHashtable $filter | Select-Object -ExpandProperty Message
A handle to an object was requested.
Subject:
    Security ID:      S-1-5-21-2318445812-3516008893-216915059-1002
    Account Name:     user
    Account Domain:   GRAPHITE
    Logon ID:         0x524D0

Object:
    Object Server:     Security
    Object Type:       Mutant
    Object Name:       \Sessions\2\BaseNamedObjects\ABC
    Handle ID:        0xfb4
    Resource Attributes: -

Process Information:
    Process ID:       0xaac
```

Process Name: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

Access Request Information:

Transaction ID: {00000000-0000-0000-0000-000000000000}

Accesses: DELETE
 READ_CONTROL
 WRITE_DAC
 WRITE_OWNER
 SYNCHRONIZE
 Query mutant state

Access Reasons: -

Access Mask: 0x1F0001

Privileges Used for Access Check: -

Restricted SID Count: 0

Listing 9-10: Viewing the open audit event for the Mutant object

We first set up a filter for the security event log and event ID 4656, which corresponds to the opening of a handle. We then use the filter with `Get-WinEvent` and select the event's textual message.

The output begins with this textual description of the event, which confirms that it was generated in response to a handle being opened. After this comes the `Subject`, which includes the user's information, including their SID and username. To look up the username, the kernel sends the audit event to the LSASS process.

Next are the details of the opened object. These include the object server (`Security`, representing the SRM), the object type (`Mutant`), and the native path to the object, as well as the handle ID (the handle number for the object). If you query the handle value returned from the `NtCreateMutant` system call, it should match this value. We then get some basic process information, and finally some information about the access granted to the handle.

How can we distinguish between success and failure events? The best way to do this is to extract the `KeywordsDisplayNames` property, which contains either `Audit Success` if the handle was opened or `Audit Failure` if the handle could not be opened. **Listing 9-11** shows an example.

```
PS> Get-WinEvent -FilterHashtable $filter | Select-Object KeywordsDisplayNames
KeywordsDisplayNames
-----
{Audit Success}
{Audit Failure}
--snip--
```

Listing 9-11: Extracting KeywordsDisplayNames to view the success or failure status

When you close the handle to the object you'll get another audit event, with the event ID 4658, as shown in [Listing 9-12](#).

```
PS> $filter = @{logname = 'Security'; id = @(4658)}
PS> Get-WinEvent -FilterHashtable $filter | Select-Object -ExpandProperty Message
The handle to an object was closed.
Subject :
    Security ID:      S-1-5-21-2318445812-3516008893-216915059-1002
    Account Name:     user
    Account Domain:   GRAPHITE
    Logon ID:         0x524D0

Object:
    Object Server:    Security
    Handle ID:        0xfb4

Process Information:
    Process ID:       0xaac
    Process Name:     C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
```

Listing 9-12: Viewing the audit event generated when the Mutant object handle is closed

You might notice that the information provided about the closing of the object handle is slightly less detailed than the information generated when the handle was opened. You can manually correlate the open and close handle events by using the handle IDs, which should match.

It's possible to generate object audit events manually from user mode using some additional system calls. However, to do so you need the

SeAuditPrivilege privilege, which is typically only granted to the *SYSTEM* account, not to normal administrators.

You can generate the audit event at the same time as an access check using the `NtAccessCheckAndAuditAlarm` system call, which has all the same object ACE variants as the normal access checks do. You can access it using the `Get-NtGrantedAccess` PowerShell command with the `Audit` parameter.

You can also generate events manually using the `NtOpenObjectAuditAlarm` and `NtCloseObjectAuditAlarm` system calls, which PowerShell exposes through the `Write-NtAudit` command. Run the commands in [Listing 9-13](#) as the *SYSTEM* user to manually generate audit log events.

```
❶ PS> Enable-NtTokenPrivilege SeAuditPrivilege -WarningAction Stop
PS> $owner = Get-NtSid -KnownSid Null
PS> $sd = New-NtSecurityDescriptor -Type Mutant -Owner $owner -Group $owner
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Access GenericAll
    -MapGeneric
❷ PS> Add-NtSecurityDescriptorAce $sd -Type Audit -Access GenericAll
    -Flags SuccessfulAccess, FailedAccess -KnownSid World -MapGeneric
❸ PS> $handle = 0x1234
❹ PS> $r = Get-NtGrantedAccess $sd -Audit -SubsystemName "SuperSecurity"
    -ObjectName "Badger" -ObjectCreation -HandleId $handle -PassResult
❺ PS> Write-NtAudit -Close -SubsystemName "SuperSecurity" -HandleId $handle
    -GenerateOnClose:$r.GenerateOnClose
```

Listing 9-13: Manually generating audit log events

We start by enabling `SeAuditPrivilege` ❶, as otherwise the rest of the script will fail. This privilege must be enabled on the primary token; you can't impersonate a token with the privilege, which is why you must run the PowerShell instance as the *SYSTEM* user.

After enabling the required privilege, we build a security descriptor with a SACL to audit successful and failed access attempts ❷. We generate a fake handle ID ❸; this value would be the kernel handle in a normal audit event, but when we generate an event from user mode it can be any value we like. We can then run the access check, specifying the `Audit` pa-

parameter, which enables the other auditing parameters. We need to specify the `SubsystemName`, `ObjectTypeName`, and `ObjectName` parameters, which can be completely arbitrary. We also specify the handle ID ④.

In the output, we receive an access check result with one additional property: `GenerateOnClose`, which indicates whether we need to write a closed handle event. Calling the `Write-NtAudit` command and specifying the `close` parameter will call the `NtCloseObjectAuditAlarm` system call to generate the event. We do so, specifying the `GenerateOnClose` value from the result ⑤. If `GenerateOnClose` were `False`, we would still need to write the close event to complete the audit, but the actual close event would not be written to the audit log.

If you don't receive any audit events when you run the commands in [Listing 9-13](#), ensure that you've enabled object auditing, as we did in [Listing 9-4](#).

THE MYSTERIOUS ALARM ACE

In the list of ACE types in [Table 5-3](#), you might have noticed the `Alarm` type that is related to auditing. I mentioned in the table that the kernel does not use this type, and if you read the Microsoft technical documentation for the `Alarm` ACE type you'll see the phrase "The `SYSTEM_ALARM_ACE` structure is reserved for future use." What is its purpose, if it's always been reserved?

It's hard to tell. Kernel code checked for the `Alarm` ACE type starting in Windows NT 3.1, until Microsoft removed the check in Windows XP. The Windows developers even defined `AlarmCallback`, `AlarmObject`, and `AlarmObjectCallback` variants, though code doesn't seem to have checked these in the Windows 2000 kernel, where object ACEs were introduced. It is clear from old kernels that the `Alarm` ACE type was handled; less clear is whether an `Alarm` ACE could generate an event to be monitored. Even in the documentation for versions of Windows that handled the `Alarm` ACE type, it is marked as unsupported.

As to what the `Alarm` ACE might have done, it's likely a holdover from Windows NT's VMS roots. VMS had a similar security model to Windows NT, including the use of ACLs and ACEs. In VMS, audit ACEs are written to an audit logfile,

as on Windows, and the alarm ACEs would generate real-time ephemeral security events in the system console or an operator's terminal once a user enabled alarms using the `REPLY/ENABLE=SECURITY` command. It's likely that Microsoft added support for this ACE type to the Windows kernel but never implemented the ability to send these real-time events. With modern logging alternatives such as Event Tracing for Windows (ETW), which provides much more comprehensive security information in real time, the chances of Microsoft reintroducing the `Alarm` ACE (or implementing its variants) in the future are slim.

Configuring the Global SACL

Correctly configuring the SACL for every resource can be difficult, as well as time-consuming. For this reason, the advanced audit policy allows you to configure a global SACL for files or registry keys. The system will use this global SACL if no SACL exists for a resource, and for resources that already have a SACL, it will merge the global and resource SACLs. Because these broad auditing configurations can swamp your logging output and impede your ability to monitor events, I recommend that you use global SACLs sparingly.

You can query the global SACL by specifying either the `File` or `Key` value to the `GlobalSacl` parameter of the `Get-NtAuditSecurity` PowerShell command. You can also modify the global SACL with the `Set-NtAuditSecurity` command, specifying the same `GlobalSacl` parameter. To test this behavior, run the commands in [Listing 9-14](#) as an administrator.

```
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> $sd = New-NtSecurityDescriptor -Type File
PS> Add-NtSecurityDescriptorAce $sd -Type Audit -KnownSid World
    -Access WriteData -Flags SuccessfulAccess
PS> Set-NtAuditSecurity -GlobalSacl File -SecurityDescriptor $sd
PS> Get-NtAuditSecurity -GlobalSacl File |
    Format-NtSecurityDescriptor -SecurityInformation Sacl -Summary
<SACL>
Everyone: (Audit)(SuccessfulAccess)(WriteData)
```

Listing 9-14: Setting and querying the global file SACL

We start by building a security descriptor containing a SACL with a single Audit ACE. We then call `Set-NtAuditSecurity` to set the global SACL for the `File` type. Finally, we query the global SACL to make sure it's set correctly.

You can remove the global SACL by passing a security descriptor with a NULL SACL to `Set-NtAuditSecurity`. To create this security descriptor, use the following command:

```
PS> $sd = New-NtSecurityDescriptor -NullSacl
```

Worked Examples

Let's wrap up with some worked examples that use the commands you learned about in this chapter.

Verifying Audit Access Security

When you're checking whether malicious code has compromised an untrusted Windows system, it's a good idea to verify that the security settings haven't been modified. One check you might want to perform is determining whether a non-administrator user has the access needed to change the audit policy on the system. If a non-administrator user can change the policy, they could disable auditing and hide their access to sensitive resources.

We can inspect the audit policy's security descriptor manually, or do so using the `Get-NtGrantedAccess` PowerShell command. Run the commands in [Listing 9-15](#) as an administrator.

```
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> $sd = Get-NtAuditSecurity
PS> Set-NtSecurityDescriptorOwner $sd -KnownSid LocalSystem
PS> Set-NtSecurityDescriptorGroup $sd -KnownSid LocalSystem
PS> Get-NtGrantedAccess $sd -PassResult
Status          Granted Access Privileges
-----
STATUS_SUCCESS  GenericRead      NONE

PS> Use-NtObject($token = Get-NtToken -Filtered -Flags LuaToken) {
```

```

Get-NtGrantedAccess $sd -Token $token -PassResult
}
Status                Granted Access Privileges
-----
STATUS_ACCESS_DENIED 0                NONE

```

Listing 9-15: Performing an access check on the audit policy security descriptor

We start by querying for the audit policy security descriptor and setting the `Owner` and `Group` fields. These fields are required for the access check process, but the security descriptor returned from the `Get-NtAuditSecurity` command does not contain them.

We can then pass the security descriptor to the `Get-NtGrantedAccess` command to check it against the current administrator token. The result indicates the caller has `GenericRead` access to the audit policy, which allows them to query the policy but not set it without enabling `SeSecurityPrivilege`.

Finally, we can remove the *Administrators* group from the token by creating a filtered token with the `LuaToken` flag. Running the access check with the filtered token indicates that it has no granted access to the audit policy (not even read access). If this second check returns a status other than `STATUS_ACCESS_DENIED`, you can conclude that the default audit policy security descriptor has been changed, and it's worth checking whether this was done intentionally or maliciously.

Finding Resources with Audit ACEs

Most resources aren't configured with a SACL, so you might want to enumerate the resources on the system that have one. This can help you understand what resources might generate audit log events. [Listing 9-16](#) provides a simple example in which we find these resources. Run the commands as an administrator.

```

PS> Enable-NtTokenPrivilege SeDebugPrivilege, SeSecurityPrivilege
❶ PS> $ps = Get-NtProcess -Access QueryLimitedInformation, AccessSystemSecurity
    -FilterScript {
    ❷ $sd = Get-NtSecurityDescriptor $_ -SecurityInformation SACL

```

```
$sd.HasAuditAce
}
❸ PS> $ps | Format-NtSecurityDescriptor -SecurityInformation Sacl
Path: \Device\HarddiskVolume3\Windows\System32\lsass.exe
Type: Process
Control: SaclPresent

<SACL>
- Type   : Audit
- Name   : Everyone
- SID    : S-1-1-0
- Mask   : 0x00000010
❹ - Access: VmRead
- Flags  : SuccessfulAccess, FailedAccess

PS> $ps.Close()
```

Listing 9-16: Finding processes with configured SACLs

We focus on `Process` objects here, but you can apply this same approach to other resource types.

We first open all processes for `QueryLimitedInformation` and `AccessSystemSecurity` access ❶. We apply a filter to the processes, querying for the SACL from the `Process` object, then returning the value of the `HasAuditAce` property ❷. This property indicates whether the security descriptor has at least one audit ACE.

We then pipe the results returned from the `Get-NtProcess` command into `Format-NtSecurityDescriptor` to display the SACLs ❸. In this case, there is only a single entry, for the LSASS process. We can see that the audit ACE logs an event whenever the LSASS process is opened for `VmRead` access ❹.

This policy is a default audit configuration on Windows, used to detect access to the LSASS process. The `VmRead` access right allows a caller to read the virtual memory of a process, and this ACE aims to detect the extraction of the LSASS memory contents, which can include passwords and other authentication credentials. If the process is opened for any other access right, no audit log entry will be generated.

Wrapping Up

In this chapter, we covered the basics of security auditing. We started with a description of the security event log and the types of log entries you might find when auditing resource access. Next, we looked at configuring the audit policy and setting advanced audit policies with the `set-NtAuditPolicy` command. We also discussed how Windows controls access to the audit policy and the importance of the `SeSecurityPrivilege` privilege, used for almost all audit-related configuration.

To enable auditing on an object, we must modify the SACL to define rules for generating the events enabled by the policy. We walked through examples of generating audit events automatically, using the SACL, and manually, during a user-mode access check.

We've now covered all aspects of the SRM: security access tokens, security descriptors, access checking, and auditing. In the rest of this book, we'll explore the various mechanisms to authenticate to a Windows system.